Pablo Jose Lorenzo Arrivillaga
1530324
German Archila
1526824

Análisis de Arquitectura de Seguridad y Control de Acceso

1. Selección del Mecanismo de Autenticación

A continuación se evalúan las tres alternativas propuestas bajo los requerimientos de Eventia:

Mecanismo	Ventajas Clave	Limitaciones en Eventia	Complejidad
Sesiones Servidor (Cookies HttpOnly)	Revocación inmediata centralizada. Inmune a XSS si se usa HttpOnly. Almacenamiento seguro nativo.	Requiere estado centralizado en servidor (Redis/DB) al escalar. Acoplamiento estricto cliente-servidor.	Baja
Tokens JWT (Stateless)	Arquitectura Decoupled/API First. Escalabilidad horizontal sin estado. Estándar para clientes SPA/Móviles.	Complejidad para revocación inmediata antes de expirar. Riesgo de XSS si se almacena en localStorage.	Media
Federación OpenID Connect (OIDC)	Excelente experiencia de usuario (Social Login). Sin gestión de contraseñas locales. Alta confianza.	Fricción si el usuario no tiene cuenta de proveedor. Dependencia de terceros para validación MVP.	Alta

Respuestas Puntuales de Selección Técnica:

1. ¿Qué alternativa utilizarían para la versión actual de Eventia?

Se selecciona **Autenticación mediante Tokens JWT (Access Tokens de vida corta + Refresh Tokens) transportados mediante Cookies HttpOnly, Secure y SameSite=Lax**.

2. ¿Por qué consideran que es adecuada para su arquitectura?

Para una startup en fase de validación con equipo pequeño y stack moderno (SPA Frontend + REST API Backend NestJS), los JWT eliminan la necesidad de consultar la base de datos central en cada petición HTTP para verificar la sesión activa, reduciendo la latencia. Transportar el JWT dentro de cookies `HttpOnly` resuelve el mayor riesgo de JWT en navegadores (robo por scripts XSS).

3. Ventajas y Limitaciones:

- **Ventajas:** Arquitectura API Stateless pura, escalable, rápida ejecución en NestJS mediante Guards, y mitigación total de XSS al no exponer el token al DOM de JavaScript.
- **Limitaciones:** Si un usuario es bloqueado o cambia de rol, los cambios surten efecto hasta que vence el Access Token (mitigado con un tiempo de vida corto de 15 minutos).

4. ¿Cómo se mantendría la autenticación entre solicitudes?

El cliente enviará automáticamente la Cookie en cada petición HTTPS. El servidor (NestJS JWT Guard) extraerá la cookie, verificará la firma criptográfica (HMAC-SHA256 o RSA), confirmará la no expiración (`exp`) e inyectará el Payload (`sub`, `email`, `role`) en el objeto `request.user`.

Pablo Jose Lorenzo Arrivillaga
1530324
German Archila
1526824

5. ¿Cómo manejarían el cierre de sesión y la expiración de las credenciales?

A través de un mecanismo dual (Access token + refresh Token), en donde el Access token tiene una vida corta (15 min) y un refresh token con una vida prolongada (7 dias) guardado de forma segura en la base de datos con un hash bcrypt/argon2.